

INTRODUCTION

1.INTRODUCTION

1.1 Introduction

The cable television industry in India serves millions of households across urban and rural areas. With rapid digitization mandated by the government, cable TV operators are transitioning from analog to digital broadcasting, significantly increasing their operational complexity. Managing subscriber data, payment collections, outstanding dues, and regulatory compliance requires modern digital tools.

CablePro is a purpose-built, web-based Cable TV Management System designed to address the operational challenges faced by local cable TV operators. The system provides a centralized platform where operators can manage all aspects of their cable TV business — from adding new subscribers to tracking monthly payments and generating financial reports.

Traditional cable TV operators maintain subscriber information in physical registers or simple spreadsheet files. These methods are error-prone, time-consuming, and do not provide real-time insights into business performance. CablePro replaces these legacy approaches with a modern, mobile-first digital solution that provides instant access to business data from any device.

The application is designed as a Progressive Web App (PWA) that works seamlessly on smartphones, tablets, and desktops. This is particularly important for cable TV operators who need to access subscriber information and record payments while visiting customers in the field.

The name 'CablePro' reflects the professional-grade nature of the system. The 'Pro' suffix indicates that this is not just a basic contact list but a comprehensive business management platform with sophisticated features such as plan duration tracking, automatic outstanding calculation, payment method analytics, and cloud backup.

CablePro integrates with Firebase Authentication for secure user management, supports Google OAuth for convenient sign-in, and uses the Google Drive API for cloud data backup and recovery. The application stores operational data locally in the browser for fast performance and offline capability, while providing cloud synchronization for data protection.

1.2 Objective of the Project

The primary objective of CablePro is to develop a comprehensive, user-friendly, and technologically modern management system for cable TV operators. The system is designed to achieve the following specific objectives:

- **Centralized Subscriber Management:** Provide a single platform for managing all subscriber (customer) information including personal details, STB numbers, plan details, and connection status.
- **Automated Outstanding Calculation:** Automatically calculate outstanding dues for each subscriber based on their plan duration and payment history, eliminating manual calculations.
- **Payment Tracking and Analytics:** Record and track all payment transactions with support for multiple payment methods (Cash and UPI), and provide real-time payment analytics through visual charts.
- **Financial Reporting:** Generate detailed financial reports with daily, weekly, and monthly breakdowns, exportable as PDF and CSV files for accounting purposes.
- **Mobile-First Design:** Ensure the application works flawlessly on mobile devices since operators frequently need to access data in the field while collecting payments.
- **Cloud Data Backup:** Provide secure data backup to Google Drive and restore capability to protect against data loss.
- **Secure Authentication:** Implement multi-method authentication including email/password and Google OAuth to ensure only authorized operators access the system.
- **Dark Mode Support:** Provide a dark mode interface option for comfortable use in low-light environments.

- Data Import/Export: Support CSV import and export for bulk customer data management and migration from existing systems.
- Real-Time Dashboard: Display real-time business metrics on a dashboard with visual analytics for quick business insights.

1.3 Purpose of the Project

The purpose of CablePro is to modernize and digitize the operations of cable TV operators by replacing manual, paper-based processes with an efficient digital management system. The project addresses several key purposes:

Operational Efficiency: Manual subscriber management is inefficient and error-prone. CablePro automates routine tasks such as outstanding due calculation, payment status updates, and report generation, significantly reducing the time operators spend on administrative work.

Business Visibility: Small cable TV operators often lack visibility into their business performance. CablePro's dashboard provides real-time metrics on total customers, active subscribers, outstanding collection, and payment trends, enabling data-driven business decisions.

Payment Collection Improvement: By clearly showing which customers have outstanding dues and enabling easy payment recording with customer search functionality, CablePro helps operators improve their payment collection rates.

Data Security and Backup: Physical registers are vulnerable to loss, damage, and theft. CablePro stores data digitally with cloud backup to Google Drive, protecting the operator's valuable business data.

Professional Record-Keeping: CablePro generates professional financial reports that can be used for business planning, tax compliance, and stakeholder communication.

Accessibility and Mobility: The mobile-first PWA design allows operators to access their subscriber database and record payments from anywhere, using any device with a web browser.

Scalability: The system is designed to handle growing subscriber bases without performance degradation, accommodating business growth without requiring system replacement.

1.4 Scope of the Project

CablePro covers all major operational aspects of cable TV subscriber management. The scope of the project includes:

1.4.1 User Authentication and Profile Management

The system supports multiple authentication methods including email/password login, Google OAuth, and a built-in operator account. User profile management allows operators to update their display name and profile picture. Account deletion with a 7-day grace period and account restoration are also supported.

1.4.2 Customer (Subscriber) Management

Complete lifecycle management of cable TV subscribers including adding new customers with STB number, contact details, address, and plan information. The system supports plan durations of 1, 3, 6, and 12 months with automatic billing cycle tracking. Customers can be filtered by status (All, Active, Deactive, Overdue, Paid) and searched by name, phone number, or STB number.

1.4.3 Payment Management

Recording and tracking of all payment transactions with support for Cash and UPI payment methods. The payment module shows today's collection, weekly collection, and monthly

collection totals. Payments can be confirmed, cancelled, or marked as pending. Duplicate payment prevention based on billing cycle is implemented.

1.4.4 Financial Reporting

Comprehensive financial reporting with month-wise and custom date range filtering. Reports include total collection, transaction count, average transaction value, collection trend charts, and payment method breakdown. Reports can be exported as PDF (via browser print dialog) and CSV files.

1.4.5 Dashboard Analytics

Real-time business dashboard displaying total customers, active customers, outstanding balance, and paid customers count. Weekly collection bar charts and payment method distribution pie charts provide visual business insights. Quick action shortcuts for common tasks are displayed prominently.

1.4.6 Data Backup and Recovery

Google Drive integration for cloud backup and restore of all business data including customers, payments, and settings. Configurable backup frequency options (Manual, Every 3 Days, Weekly, Monthly) with automatic backup reminders.

1.4.7 Data Import/Export

CSV import functionality for bulk customer data upload, useful for migrating from existing systems. CSV export of customer and payment data for external analysis.

1.5 Significance of the Project

CablePro holds significant importance in the context of India's cable TV digitization program. The Telecom Regulatory Authority of India (TRAI) has mandated the digitization of all cable TV connections, requiring operators to maintain detailed subscriber records and manage set-

top box (STB) allocations. CablePro directly addresses this regulatory requirement by providing a structured subscriber database with STB number tracking.

The project is significant for the following reasons:

- **Economic Impact:** Improved payment collection through automated outstanding tracking can significantly increase operator revenues. Studies suggest that digital billing systems improve collection rates by 15-25% compared to manual methods.
- **Digital India Initiative:** The project aligns with the Government of India's Digital India initiative by digitizing operations of thousands of small cable operators who are the backbone of India's last-mile connectivity.
- **Technology Accessibility:** By building the application as a PWA accessible through any modern web browser, CablePro makes advanced management technology accessible to operators who cannot afford expensive proprietary software solutions.
- **Data-Driven Operations:** The analytics and reporting features enable operators to make informed business decisions based on actual data rather than intuition, improving business outcomes.
- **Academic Significance:** This project demonstrates the application of modern web technologies including React, TypeScript, Next.js, Firebase, and Google APIs in solving real-world business problems, serving as a practical example of full-stack web development.

SYSTEM SPECIFICATION

2.SYSTEM SPECIFICATION

2.1 Hardware Specifications

The following table outlines the minimum and recommended hardware specifications for running CablePro effectively:

Component	Minimum Requirements	Recommended
Processor	Intel Core i3 / AMD Ryzen 3 (1.8 GHz)	Intel Core i5 / AMD Ryzen 5 (2.5 GHz+)
RAM	4 GB	8 GB or higher
Storage	256 MB free space (browser cache)	1 GB for development environment
Display	1280 x 720 px (HD)	1920 x 1080 px (Full HD)
Network	2 Mbps broadband	10 Mbps or higher
Mobile Device	Android 8.0 / iOS 13+	Android 11 / iOS 15+
Browser	Any modern browser (2019+)	Google Chrome 90+ / Safari 15+

Note: Since CablePro is a web-based application, the hardware requirements primarily apply to the device used to access the application. The application does not require dedicated server hardware as it runs entirely in the browser with Firebase as the backend-as-a-service.

2.1.1 Development Hardware

The following hardware was used during the development and testing of CablePro:

Component	Specification
Processor	Intel Core i7 (12th Gen)
RAM	16 GB DDR5
Storage	512 GB NVMe SSD
Display	1920 x 1080 Full HD
Operating System	Windows 11 / macOS Ventura

2.2 Software Specifications

2.2.1 Development Environment

Software	Version / Details
Node.js	v18.17.0 LTS (Runtime Environment)
npm	v9.6.7 (Package Manager)
Next.js	14.x with App Router
TypeScript	5.x (Static Typing)
React	18.x
Tailwind CSS	4.x (Utility-First CSS)
Firebase SDK	v10.x
VS Code	Latest version (IDE)
Git	2.x (Version Control)
GitHub	Source Code Repository

2.2.2 Production/Deployment Environment

Service	Details
Hosting Platform	Vercel (Next.js optimized hosting)
Authentication	Firebase Authentication
Database	Browser localStorage (primary), Firebase Firestore (optional)
Cloud Backup	Google Drive API v3
CDN	Vercel Edge Network
SSL/TLS	Automatic HTTPS via Vercel
Domain	cablepro.in (custom domain)

2.2.3 Browser Support

Browser	Minimum Version	Support Level
Google Chrome	90+	Full Support
Microsoft Edge	90+	Full Support
Mozilla Firefox	88+	Full Support
Apple Safari	14+	Full Support
Samsung Internet	14+	Full Support
Opera	76+	Full Support

TECHNOLOGIES USED

3. TECHNOLOGIES USED

3.1 Front-End Technologies

3.1.1 Next.js 14 with App Router

Next.js is a React-based full-stack web framework developed by Vercel. CablePro uses Next.js 14 with the App Router (app directory structure), which provides file-system based routing, server-side rendering (SSR), static site generation (SSG), and API routes. The App Router represents the latest paradigm in React application architecture, supporting React Server Components (RSC) alongside client-side components.

Key Next.js features used in CablePro include: file-system routing for page navigation, 'use client' directive for interactive components, Next.js Image component for optimized image loading, and built-in font optimization with Google Fonts.

3.1.2 React 18

React is the industry-standard JavaScript library for building user interfaces. CablePro uses React 18, which introduces concurrent rendering features, automatic batching of state updates, and the new `useTransition` and `useDeferredValue` hooks for improved UI responsiveness. All interactive components in CablePro are React functional components using hooks for state management.

3.1.3 TypeScript

TypeScript is a statically-typed superset of JavaScript that adds optional type annotations. CablePro uses TypeScript throughout the codebase to ensure type safety, catch errors during development, and improve code maintainability. Custom TypeScript interfaces define the data models for Customer, Payment, and User entities.

3.1.4 Tailwind CSS v4

Tailwind CSS is a utility-first CSS framework that provides low-level CSS utility classes for styling. CablePro uses Tailwind CSS v4 with a custom theme configuration that defines the color palette (primary blue #404fb5, success green, orange, teal), spacing scale, and dark mode variant. The responsive design is achieved entirely through Tailwind's responsive prefixes (sm:, lg:) without writing custom CSS.

3.1.5 Recharts

Recharts is a React-based charting library built on top of SVG. CablePro uses Recharts for the following visualizations: BarChart for weekly payment collection display on the dashboard, PieChart for payment method (Cash vs UPI) distribution, and AreaChart for the collection trend in the Reports module. All charts are responsive and support tooltip interactions.

3.1.6 Material Symbols (Google Icons)

Material Symbols is Google's modern icon library, loaded via Google Fonts CDN. CablePro uses the Outlined variant of Material Symbols for consistent iconography throughout the application. Icons are used for navigation items, action buttons, status indicators, and form field prefixes.

3.2 Back-End Technologies

3.2.1 Firebase Authentication

Firebase Authentication is Google's managed authentication service. CablePro integrates Firebase Auth to support multiple sign-in methods: Email/Password authentication for direct login, Google OAuth for convenient one-click sign-in, and password reset via email. Firebase Auth handles all security concerns including token management, session persistence, and brute force protection.

3.2.2 Firebase/Firestore (Optional Integration)

Firebase Firestore is a NoSQL document database with real-time synchronization capabilities. While CablePro primarily uses browser `localStorage` for data storage to enable offline functionality, the Firebase integration is structured to support optional Firestore sync for multi-device data access.

3.2.3 Google Drive API v3

The Google Drive API is used for cloud data backup and restore functionality. CablePro requests the 'drive.file' scope (limited access to files created by the app) to store a backup JSON file in the user's Google Drive under a 'Cable Pro Backup' folder. The API is accessed using the OAuth access token obtained during Google Sign-In.

3.2.4 Browser localStorage

`localStorage` is a Web Storage API that provides persistent key-value storage in the browser. CablePro uses `localStorage` as the primary data store for customer records, payment history, user preferences (theme), and backup settings. Data is namespaced by user ID to support multiple operators on the same device. The `localStorage` implementation includes data sanitization (removing legacy keys) and schema migrations for backward compatibility.

3.3 Development Tools

3.3.1 Visual Studio Code

VS Code is used as the primary Integrated Development Environment (IDE). Extensions used include ESLint for code linting, Prettier for code formatting, TypeScript Language Service for type checking, and the Tailwind CSS IntelliSense extension for class autocompletion.

3.3.2 Git and GitHub

Git is used for version control with the project repository hosted on GitHub. The main branch contains the production code. Feature branches are used for developing new functionality before merging to main.

3.3.3 Vercel

Vercel is the deployment platform for CablePro. It provides automatic deployments from the GitHub repository, preview deployments for pull requests, edge network CDN, automatic HTTPS, and Next.js-specific optimizations. Environment variables for Firebase configuration are managed through Vercel's environment variable system.

3.3.4 npm (Node Package Manager)

npm is used for managing JavaScript dependencies. Key dependencies include docx (document generation), recharts (charting), and the Firebase SDK. Development dependencies include TypeScript compiler, ESLint, and PostCSS for Tailwind processing.

Category	Technology	Purpose
Framework	Next.js 14	React Framework with App Router
UI Library	React 18	Component-based UI
Language	TypeScript	Static Type Safety
Styling	Tailwind CSS v4	Utility-first CSS
Charts	Recharts	Data Visualization
Auth	Firebase Authentication	User Management
Storage	localStorage / Firebase	Data Persistence
Backup	Google Drive API v3	Cloud Backup
Hosting	Vercel	Deployment & CDN
Icons	Material Symbols	UI Iconography

SYSTEM ANALYSIS

4.SYSTEM ANALYSIS

4.1 Existing System

The existing methods used by cable TV operators for subscriber management range from completely manual approaches to basic spreadsheet tools. An analysis of the existing systems reveals the following:

4.1.1 Paper-Based Register System

The most common method used by small cable operators is maintaining a physical register with subscriber information. Each page in the register typically contains the subscriber's name, address, connection date, STB number, monthly charges, and payment dates. Operators manually cross-reference the register to identify outstanding dues.

Drawbacks of Paper-Based Systems:

- Data loss risk due to fire, water damage, or physical deterioration of registers
- Difficult to search for specific subscriber information without manual page-by-page scanning
- No automatic calculation of outstanding dues
- Cannot be accessed remotely or by multiple staff members simultaneously
- Report generation requires manual addition and calculation of figures
- No backup mechanism exists
- Prone to human errors in recording and calculation

4.1.2 Spreadsheet-Based System

Some operators use Microsoft Excel or Google Sheets to maintain subscriber records. While better than paper registers, spreadsheets present their own challenges:

- No dedicated features for cable TV management (STB tracking, plan duration, etc.)
- Complex formula setup required for automatic calculations
- Not optimized for mobile use, making field data entry difficult
- No built-in authentication or access control
- Data sharing between multiple devices requires manual file transfer or cloud storage setup
- No integrated payment tracking with payment method analytics
- Cannot generate formatted reports or receipts

4.1.3 Proprietary Cable Management Software

Some larger operators use proprietary cable TV billing software. However, these solutions have significant drawbacks for small operators:

- High licensing costs (often Rs. 10,000 - 50,000 per year)
- Require dedicated Windows PCs, not mobile-friendly
- Complex installation and maintenance requirements
- Poor user experience and outdated interfaces
- Limited or no cloud backup functionality
- Vendor lock-in for data and features

4.2 Proposed System

CablePro addresses all the limitations of existing systems by providing a modern, web-based cable TV management platform. The proposed system offers the following improvements:

4.2.1 Advantages of CablePro

- **Accessibility:** Works on any device with a web browser — smartphones, tablets, laptops, and desktops. No installation required.

- **Mobile-First Design:** Optimized for mobile use with touch-friendly interfaces and PWA capability, allowing operators to record payments and check subscriber status in the field.
- **Automatic Calculations:** Outstanding dues are automatically calculated based on payment history and plan duration (1, 3, 6, or 12 months), eliminating manual computation.
- **Secure Authentication:** Firebase-powered authentication with email/password and Google OAuth, ensuring only authorized personnel access the system.
- **Cloud Backup:** Automatic and manual backup to Google Drive protects data against device loss or failure.
- **Real-Time Analytics:** Dashboard provides instant visibility into business performance with charts and KPI metrics.
- **Report Generation:** One-click generation of PDF and CSV reports for accounting and business planning.
- **Search and Filter:** Quick search by name, phone, or STB number, and filter customers by payment status.
- **Cost-Effective:** Free to use with no licensing fees, accessible through any browser.
- **Duplicate Prevention:** System prevents duplicate STB numbers and phone numbers, ensuring data integrity.

4.2.2 Comparison: Existing vs. Proposed System

Feature	Existing System	CablePro (Proposed)
Access	Single location/device	Any device, anywhere
Mobile Support	None / Poor	Fully mobile-optimized
Data Entry	Manual, error-prone	Guided forms with validation
Outstanding Calc.	Manual calculation	Automatic calculation
Search	Manual scanning	Instant search by name/phone/STB
Reporting	Manual summarization	Automated with export
Backup	None	Google Drive cloud backup

Analytics	None	Real-time charts & KPIs
Cost	Rs. 0 - 50,000/yr	Free to use
Security	Physical lock	Firebase Auth with OAuth
Multi-Device	Not supported	Fully supported

4.3 Feasibility Study

4.3.1 Technical Feasibility

CablePro is technically feasible as it uses proven, widely-adopted technologies. Next.js, React, TypeScript, Tailwind CSS, and Firebase are all production-ready technologies used by major corporations. The development team has expertise in these technologies, and extensive documentation, community support, and third-party libraries are available. Browser localStorage provides reliable local data storage without requiring server infrastructure.

4.3.2 Operational Feasibility

The system is operationally feasible as cable TV operators already use smartphones and basic computer applications. The intuitive interface design minimizes the learning curve. The mobile-first approach ensures operators can adopt the system without purchasing new hardware. The PWA capability means the application works even with intermittent internet connectivity.

4.3.3 Economic Feasibility

CablePro is economically feasible for operators of all sizes. The application can be deployed on Vercel's free tier for small operators and upgraded to paid plans as usage grows. Firebase Authentication's free tier (Spark plan) supports up to 10,000 authentications per month, sufficient for most cable operators. Development costs are minimal as the project uses open-source technologies with no licensing fees.

SYSTEM DESIGN

5.SYSTEM DESIGN

5.1 Architecture Overview

CablePro follows a modern client-server architecture with the client-side React application serving as the primary interface layer, Firebase as the authentication and backend service layer, browser localStorage as the local data persistence layer, and Google Drive API as the cloud backup layer.

5.1.1 Application Architecture

The application is structured as a Next.js App Router application with the following directory structure:

Directory/File	Purpose
src/app/(app)/	Protected application routes (requires authentication)
src/app/(app)/dashboard/	Dashboard page with analytics
src/app/(app)/customers/	Customer list and management
src/app/(app)/customers/view/	Individual customer detail view
src/app/(app)/payments/	Payment list and management
src/app/(app)/payments/new/	New payment recording form
src/app/(app)/reports/	Financial reports and analytics
src/app/(app)/settings/	Application and user settings
src/app/page.tsx	Login page (unauthenticated)
src/app/signup/	New operator registration page
src/lib/store.tsx	Global state management (React Context)
src/lib/types.ts	TypeScript type definitions
src/lib/firebase.ts	Firebase configuration and exports
src/components/AppShell.tsx	Main application layout with navigation
src/components/Modal.tsx	Reusable modal dialog component

5.1.2 State Management Architecture

CablePro uses React Context API for global state management, implemented in `src/lib/store.tsx`. The store provides the following state and actions to all application components:

- Authentication State: `currentUser`, `login`, `logout`, `firebaseEmailSignup`, `firebaseEmailLogin`, `googleLogin`, `deleteAccount`, `restoreAccount`
- Customer State: `customers` array, `addCustomer`, `updateCustomer`, `deleteCustomer`, `getCustomer`, `importCustomers`
- Payment State: `payments` array, `addPayment`, `updatePayment`, `getPaymentsForCustomer`
- Computed Metrics: `totalCustomers`, `activeCustomers`, `totalPaidCustomers`, `totalOutstanding`, `todayCollection`, `thisMonthCollection`
- UI State: `theme` (light/dark), `showBackupReminder`, `hydrated`
- Backup State: `backupFrequency`, `googleAccessToken`, `backupToDrive`, `restoreFromDrive`

5.2 Data Flow Description

5.2.1 Customer Management Flow

When an operator adds a new customer, the form data passes through the `addCustomer` function in the store, which first validates the customer data (checking for duplicate phone numbers and STB numbers, validating 10-digit phone format, ensuring STB number is numeric). If validation passes, a new Customer object is created with a generated ID and current timestamp. The customer is added to the `customers` array in React state, which triggers a `localStorage` save through a `useEffect` hook. The UI automatically re-renders to show the new customer.

5.2.2 Payment Recording Flow

When an operator records a payment, the system first verifies the selected customer is active. It then checks if the customer's current billing cycle is still active (based on the last payment date and plan duration) to prevent duplicate payments. If no conflict is found, the payment is added via `addPayment`, stored in state, and persisted to `localStorage`. The dashboard and customer detail view automatically reflect the updated outstanding balance.

5.2.3 Authentication Flow

On application load, the store checks `localStorage` for a saved user session (`cp_current_user` key). If found, the user is automatically logged in and their data (customers, payments) is loaded from their user-specific `localStorage` namespace (`cp_{userId}_cust` and `cp_{userId}_pay`). For new logins, Firebase Authentication validates credentials and the resulting user object is stored in both React state and `localStorage`.

5.3 Entity Relationship Description

CablePro's data model consists of three main entities: User, Customer, and Payment. The relationships between these entities are described below:

5.3.1 User Entity

The User entity represents a cable TV operator who uses the system. Key attributes include a unique ID (Firebase UID or generated), full name, email, username, role (operator or admin), and optional profile image. A user can manage multiple customers and their associated payments.

5.3.2 Customer Entity

The Customer entity represents a cable TV subscriber. Key attributes include ID, name, phone number (unique, 10 digits), address, STB number (unique, numeric), connection date, package name, monthly rate, plan duration (1/3/6/12 months), status (Active/Deactive), operator ID (foreign key to User), and soft-delete flags (`isDeleted`, `deletedAt`).

5.3.3 Payment Entity

The Payment entity records each payment transaction. Key attributes include ID, customer ID (foreign key to Customer), customer name (denormalized for efficiency), amount, payment date, payment method (Cash or UPI), billing period (month-year string), status (Confirmed/Pending/Cancelled), notes, and creation timestamp.

Relationships: One User has many Customers (one-to-many). One Customer has many Payments (one-to-many). Each Payment belongs to one Customer and indirectly to one User.

5.4 System Diagrams

5.4.1 Use Case Description

The system has a single primary actor: the Cable TV Operator. The operator can perform the following use cases:

Use Case	Actor	Description
Register Account	Operator	Create a new operator account via email/Google
Login	Operator	Authenticate using email/password or Google OAuth
View Dashboard	Operator	View business metrics and analytics
Add Customer	Operator	Register a new cable TV subscriber
Edit Customer	Operator	Update subscriber information
Delete Customer	Operator	Soft-delete a subscriber from the system
View Customer Detail	Operator	See full customer profile and payment history
Filter Customers	Operator	Filter by Active/Deactive/Overdue/Paid status
Record Payment	Operator	Record a new payment transaction
Cancel Payment	Operator	Cancel/reverse a confirmed payment
View Payments	Operator	View all payment transactions with filters

Generate Report	Operator	View financial report for a date range
Export CSV	Operator	Download payment data as CSV file
Export PDF	Operator	Print/download report as PDF
Backup to Drive	Operator	Upload data backup to Google Drive
Restore from Drive	Operator	Restore data from Google Drive backup
Import Customers	Operator	Bulk import customers from CSV file
Toggle Dark Mode	Operator	Switch between light and dark interface
Update Profile	Operator	Change display name and profile picture
Logout	Operator	Sign out from the application

TABLE STRUCTURE

6.TABLE STRUCTURE

CablePro uses TypeScript interfaces as data models and browser localStorage as the persistence layer. The following describes the structure of each data entity:

6.1 Customer Data Model

Field	Type	Description
id	string	Unique identifier (auto-generated UUID)
name	string	Customer full name (required)
phone	string	Mobile number — 10 digits, unique (required)
address	string	Customer's residential address
stbNumber	string	Set-Top Box serial number — numeric, unique (required)
connectionDate	string	Date of initial connection (YYYY-MM-DD format)
packageName	string	Name of the cable TV plan/package
monthlyRate	number	Monthly subscription rate in INR
planDuration	number	Plan validity in months (1, 3, 6, or 12)
status	string	'Active' or 'Deactive' enum value
operatorId	string	Foreign key — ID of the operator who manages this customer
createdAt	string	ISO timestamp of record creation
isDeleted	boolean?	Soft-delete flag (optional, defaults to false)
deletedAt	string?	ISO timestamp of soft-delete (optional)

6.2 Payment Data Model

Field	Type	Description
id	string	Unique identifier (auto-generated UUID)
customerId	string	Foreign key — ID of the paying customer
customerName	string	Denormalized customer name for display efficiency

amount	number	Payment amount in INR
paymentDate	string	Date of payment (YYYY-MM-DD format)
paymentMethod	string	'Cash' or 'UPI' — the method of payment
billingPeriod	string	Month-Year string (e.g., 'February 2026')
status	string	'Confirmed', 'Pending', or 'Cancelled'
notes	string	Optional operator notes about the payment
createdAt	string	ISO timestamp of record creation

6.3 User Data Model

Field	Type	Description
id	string	Firebase UID or generated UUID
username	string	Login username (for seed/legacy accounts)
password	string	Password hash (for seed accounts only)
fullName	string	Operator's display name
role	string	'operator' or 'admin' role enum
email	string	Operator's email address
profileImage	string?	Base64-encoded profile image (optional)
isDeleted	boolean?	Account deletion flag (optional)
deletedAt	string?	ISO timestamp of account deletion (optional)

6.4 localStorage Key Schema

Data is stored in the browser's localStorage using the following namespacing convention to support multiple operators on the same device:

localStorage Key	Content
cp_current_user	JSON: Currently logged-in User object
cp_{userId}_cust	JSON: Array of Customer objects for this operator
cp_{userId}_pay	JSON: Array of Payment objects for this operator
cp_{userId}_backup_freq	String: Backup frequency setting

cp_{userId}_last_backup	Number: Unix timestamp of last backup
cp_deleted_users	JSON: Map of deleted user IDs to deletion dates
cablepro_theme	String: 'light' or 'dark' theme preference

6.5 Backup Data Structure

When backing up to Google Drive, CablePro serializes all data into a JSON file (cablepro_backup.json) with the following structure:

JSON Key	Content
customers	Array of all Customer objects
payments	Array of all Payment objects
deletedUsers	Object mapping deleted user IDs to dates
settings.currentUser	Current User object
settings.backupFrequency	Backup frequency preference

6.6 Validation Rules

CablePro enforces the following validation rules for data integrity:

Field	Validation Rule	Error Message
Customer Name	Required, non-empty	Customer Name is required.
Phone Number	Required, exactly 10 digits, numeric	Mobile Number must be exactly 10 digits.
Phone Number	Unique across all customers	Mobile Number already exists for another customer.
STB Number	Required, numeric only	STB Number must be numeric only.
STB Number	Unique across all customers	STB Number already exists for another customer.
Payment Amount	Positive number greater than 0	Please enter a valid amount.
Customer (for Payment)	Must be Active status	Cannot record payment for deactive customer.

IMPLEMENTATION AND SOURCECODE

7.IMPLEMENTATION AND SOURCE CODE

7.1 TypeScript Type Definitions (src/lib/types.ts)

The types.ts file defines the core data interfaces used throughout the application. The Customer interface includes fields for id, name, phone, address, stbNumber, connectionDate, packageName, monthlyRate (in INR), planDuration (1/3/6/12 months), status ('Active' | 'Deactive'), operatorId, createdAt timestamp, and optional soft-delete fields (isDeleted, deletedAt). The Payment interface includes id, customerId, customerName (denormalized), amount, paymentDate (YYYY-MM-DD), paymentMethod ('Cash' | 'UPI'), billingPeriod (e.g., 'February 2026'), status ('Confirmed' | 'Pending' | 'Cancelled'), notes, and createdAt timestamp. The User interface includes id, username, password (for seed accounts), fullName, role ('admin' | 'operator'), email, and optional profileImage.

"

```
import { useState, useRef, useEffect } from "react";
import { useRouter } from "next/navigation";
import { useStore } from "@lib/store";
import { Customer } from "@lib/types";
import Modal from "@components/Modal";
import Image from "next/image";

export default function SettingsPage() {
  const router = useRouter();
  const { currentUser, logout, customers, payments, exportCustomersCSV,
importCustomers, updateUserProfile, deleteAccount } = useStore();
  const fileInputRef = useRef<HTMLInputElement>(null);
  const profileImageInputRef = useRef<HTMLInputElement>(null);
  const [importStatus, setImportStatus] = useState<string | null>(null);

  // Modals
  const [showLogoutConfirm, setShowLogoutConfirm] = useState(false);
  const [showDeleteConfirm, setShowDeleteConfirm] = useState(false);
  const [showEditProfile, setShowEditProfile] = useState(false);

  // Edit Profile State
  const [editName, setEditName] = useState("");
  const [editImage, setEditImage] = useState<string | undefined>(undefined);

  useEffect(() => {
    if (currentUser) {
```

```

    const timer = setTimeout(() => {
      setEditName(currentUser.fullName);
      setEditImage(currentUser.profileImage);
    }, 0);
    return () => clearTimeout(timer);
  }
}, [currentUser]);

const handleLogout = () => {
  logout();
  router.push("/");
};

const handleDeleteAccount = () => {
  deleteAccount();
  router.push("/");
};

const handleSaveProfile = () => {
  updateUserProfile({ fullName: editName, profileImage: editImage });
  setShowEditProfile(false);
};

const handleProfileImageUpload = (e: React.ChangeEvent<HTMLInputElement>) =>
{
  const file = e.target.files?.[0];
  if (!file) return;
  if (file.size > 500 * 1024) { alert("Image size must be less than 500KB"); return; }
  const reader = new FileReader();
  reader.onload = (ev) => {
    if (ev.target?.result) setEditImage(ev.target.result as string);
  };
  reader.readAsDataURL(file);
};

const handleImportCSV = (e: React.ChangeEvent<HTMLInputElement>) => {
  const file = e.target.files?.[0];
  if (!file) return;
  const reader = new FileReader();
  reader.onload = async (event) => {
    try {
      const text = event.target?.result as string;
      const lines = text.split("\n").filter(l => l.trim());
      if (lines.length < 2) { setImportStatus("✗File is empty or has no data rows.");
return; }
      const header = lines[0].split(",").map(h => h.trim().toLowerCase());
      const imported = lines.slice(1).map(line => {
        const values = line.split(",").map(v => v.trim());

```

```

    const row: Record<string, string> = {};
    header.forEach((h, i) => { row[h] = values[i] || ""; });
    return row;
  });

  const mapped: Omit<Customer, "id">[] = imported.map(row => ({
    name: row["name"] || row["customer"] || "",
    phone: row["phone"] || row["mobile"] || "",
    address: row["address"] || "Unknown",
    stbNumber: row["stb number"] || row["stb"] || "",
    connectionDate: row["user added on "] || new Date().toISOString().slice(0,
    packageName: row["package"] || "Basic",
    monthlyRate: parseFloat(row["monthly rate"] || row["plan price"] ||
row["amount"] || "0"),
    planDuration: 1,
    status: (row["status"] as "Active" | "Deactive") || "Active",
    operatorId: currentUser?.id || "u2",
    createdAt: new Date().toISOString()
  })).filter(c => c.name && c.phone);

  const res = await importCustomers(mapped as Customer[]);
  setImportStatus(`✔Successfully imported ${res.count} customers.`);
} catch {
  setImportStatus("✗Failed to parse CSV file.");
}
};
reader.readAsText(file);
// Reset input so same file can be selected again
e.target.value = "";
};

const initials = currentUser?.fullName
  ? currentUser.fullName.split(" ").map(w => w[0]).join("").toUpperCase().slice(0, 2)
  : "OP";

return (
  <div className="max-w-2xl mx-auto space-y-6">
    <h2 className="text-xl font-bold text-text-primary">Settings</h2>

    {/* Profile Section */}
    <div className="bg-white dark:bg-bg-card-dark rounded-xl shadow-sm border
border-gray-100 dark:border-border-dark overflow-hidden">
      <div className="px-5 py-4 border-b border-gray-50 dark:border-gray-800">
        <h3 className="text-base font-bold text-text-primary dark:text-
white">Profile</h3>
      </div>
      <div className="p-5 space-y-4">
        <div className="flex items-center gap-4">

```

```

        <div className="relative group cursor-pointer" onClick={() =>
setShowEditProfile(true)}>
            {currentUser?.profileImage ? (
                <Image src={currentUser.profileImage} alt="Profile" width={64}
height={64} className="w-16 h-16 rounded-full object-cover border-2 border-
primary/20" />
            ) : (
                <div className="w-16 h-16 rounded-full bg-primary/10 text-primary
flex items-center justify-center text-2xl font-bold">
                    {initials}
                </div>
            )}
            <div className="absolute inset-0 bg-black/40 rounded-full flex items-
center justify-center opacity-0 group-hover:opacity-100 transition-opacity">
                <span className="material-symbols-outlined text-white text-
sm">edit</span>
            </div>
        </div>
    </div>
    <div>
        <p className="text-lg font-bold text-text-primary dark:text-
white">{currentUser?.fullName || "User"}</p>
        <p className="text-sm text-text-secondary dark:text-gray-
400">{currentUser?.email || ""}</p>
        <p className="text-xs text-primary font-semibold mt-1
capitalize">{currentUser?.role || "operator"}</p>
    </div>
</div>
</div>
</div>
</div>

    {/* Data Summary */}
    <div className="bg-white dark:bg-bg-card-dark rounded-xl shadow-sm border
border-gray-100 dark:border-border-dark overflow-hidden">
        <div className="px-5 py-4 border-b border-gray-50 dark:border-gray-800">
            <h3 className="text-base font-bold text-text-primary dark:text-
white">Data Summary</h3>
        </div>
        <div className="divide-y divide-gray-50 dark:divide-gray-800">
            <div className="flex items-center justify-between px-5 py-4">
                <div className="flex items-center gap-3">
                    <span className="material-symbols-outlined text-
primary">group</span>
                    <span className="text-sm font-medium text-text-primary dark:text-
white">Total Customers</span>
                </div>
                <span className="text-sm font-bold text-text-primary dark:text-
white">{customers.length}</span>
            </div>
        </div>
    </div>

```

```

    <div className="flex items-center justify-between px-5 py-4">
      <div className="flex items-center gap-3">
        <span className="material-symbols-outlined text-
success">receipt_long</span>
        <span className="text-sm font-medium text-text-primary dark:text-
white">Total Payments</span>
      </div>
      <span className="text-sm font-bold text-text-primary dark:text-
white">{payments.length}</span>
    </div>
    <div className="flex items-center justify-between px-5 py-4">
      <div className="flex items-center gap-3">
        <span className="material-symbols-outlined text-
orange">storage</span>
        <span className="text-sm font-medium text-text-primary dark:text-
white">Data Storage</span>
      </div>
      <span className="text-sm font-bold text-text-secondary dark:text-gray-
400">Local (Browser)</span>
    </div>
  </div>
</div>

{/* Import / Export */}
<div className="bg-white dark:bg-bg-card-dark rounded-xl shadow-sm border
border-gray-100 dark:border-border-dark overflow-hidden">
  <div className="px-5 py-4 border-b border-gray-50 dark:border-gray-800">
    <h3 className="text-base font-bold text-text-primary dark:text-
white">Import / Export</h3>
  </div>
  <div className="p-5 space-y-4">
    <div className="flex flex-col sm:flex-row gap-3">
      <button onClick={exportCustomersCSV}
        className="flex-1 flex items-center justify-center gap-2 py-3
rounded-xl bg-primary/10 text-primary font-bold text-sm hover:bg-primary/20
transition active:scale-[0.98]">
        <span className="material-symbols-outlined text-
lg">download</span>
        Export Customers CSV
      </button>
      <button onClick={() => fileInputRef.current?.click()}
        className="flex-1 flex items-center justify-center gap-2 py-3
rounded-xl bg-teal/10 text-teal font-bold text-sm hover:bg-teal/20 transition
active:scale-[0.98]">
        <span className="material-symbols-outlined text-lg">upload</span>
        Import Customers CSV
      </button>
    </div>
  </div>
</div>

```

```

        <input ref={fileInputRef} type="file" accept=".csv" className="hidden"
onChange={handleImportCSV} />
    </div>
    {importStatus && (
        <div className="bg-bg-light dark:bg-gray-800 rounded-lg px-4 py-3
text-sm text-text-primary dark:text-white font-medium">
            {importStatus}
        </div>
    )}
    <p className="text-text-secondary dark:text-gray-400 text-xs">Export
your customer data as CSV, or import from a CSV file.</p>
</div>
</div>

    {/General Settings */}
    <div className="bg-white dark:bg-bg-card-dark rounded-xl shadow-sm border
border-gray-100 dark:border-border-dark overflow-hidden">
        <div className="px-5 py-4 border-b border-gray-50 dark:border-gray-800">
            <h3 className="text-base font-bold text-text-primary dark:text-
white">General</h3>
        </div>
        <div className="divide-y divide-gray-50 dark:divide-gray-800">
            {[
                { icon: "language", label: "Language", value: "English" },
                { icon: "currency_rupee", label: "Currency", value: "INR (₹)" },
            ].map((item, i) => (
                <div key={i} className="flex items-center justify-between px-5 py-4
hover:bg-bg-light dark:hover:bg-gray-800 transition-colors cursor-pointer">
                    <div className="flex items-center gap-3">
                        <span className="material-symbols-outlined text-text-secondary
dark:text-gray-400">{item.icon}</span>
                        <span className="text-sm font-medium text-text-primary
dark:text-white">{item.label}</span>
                    </div>
                    <div className="flex items-center gap-2">
                        <span className="text-sm text-text-secondary dark:text-gray-
400">{item.value}</span>
                        <span className="material-symbols-outlined text-gray-300
dark:text-gray-600 text-sm">chevron_right</span>
                    </div>
                </div>
            ))}
        </div>
    </div>

    {/Danger Zone */}

```

```

<div className="bg-white dark:bg-bg-card-dark rounded-xl shadow-sm border
border-danger/20 overflow-hidden">
  <div className="px-5 py-4 border-b border-danger/10">
    <h3 className="text-base font-bold text-danger">Danger Zone</h3>
  </div>
  <div className="p-5 space-y-3">
    <button onClick={() => setShowLogoutConfirm(true)}
      className="text-text-secondary dark:text-gray-400 text-sm font-bold
hover:text-danger hover:underline flex items-center gap-2">
      <span className="material-symbols-outlined text-lg">logout</span>
      Sign Out
    </button>
    <div className="pt-3 border-t border-danger/10">
      <button onClick={() => setShowDeleteConfirm(true)}
        className="text-danger text-sm font-bold hover:underline flex items-
center gap-2 w-full">
        <span className="material-symbols-outlined text-
lg">delete_forever</span>
        Delete Account
      </button>
      <p className="text-[10px] text-danger/60 mt-1 pl-7">Action is
reversible within 7 days.</p>
    </div>
  </div>
</div>

{/* Modals */}
<Modal isOpen={showLogoutConfirm} onClose={() =>
setShowLogoutConfirm(false)} title="Confirm Logout">
  <p className="text-text-secondary mb-6">Are you sure you want to log
out?</p>
  <div className="flex justify-end gap-3">
    <button onClick={() => setShowLogoutConfirm(false)} className="px-4
py-2 rounded-lg text-text-secondary dark:text-gray-400 font-semibold hover:bg-gray-
100 dark:hover:bg-gray-800">Cancel</button>
    <button onClick={handleLogout} className="px-4 py-2 rounded-lg bg-
primary text-white font-bold shadow-sm hover:bg-primary-dark">Log Out</button>
  </div>
</Modal>

<Modal isOpen={showDeleteConfirm} onClose={() =>
setShowDeleteConfirm(false)} title="Delete Account">
  <div className="bg-danger/10 text-danger p-4 rounded-xl mb-4 text-sm
font-medium">
    Warning: This will mark your account for deletion. You will have 7 days to
restore it by logging in again. After that, all data will be permanently lost.
  </div>

```

```

        <p className="text-text-primary text-sm mb-6">Type
<strong>DELETE</strong> to confirm.</p>
        /* Simplified for now, just a button */
        <div className="flex justify-end gap-3">
            <button onClick={() => setShowDeleteConfirm(false)} className="px-4 py-
2 rounded-lg text-text-secondary dark:text-gray-400 font-semibold hover:bg-gray-100
dark:dark:dark:gray-800">Cancel</button>
            <button onClick={handleDeleteAccount} className="px-4 py-2 rounded-lg
bg-danger text-white font-bold shadow-sm hover:bg-danger-dark">Delete
Account</button>
        </div>
    </Modal>

    <Modal isOpen={showEditProfile} onClose={() => setShowEditProfile(false)}
title="Edit Profile">
        <div className="space-y-4">
            <div className="flex flex-col items-center gap-3">
                <div className="relative w-24 h-24 rounded-full bg-gray-100 flex items-
center justify-center overflow-hidden cursor-pointer hover:opacity-80 transition"
                    onClick={() => profileImageInputRef.current?.click()}>
                    {editImage ? (
                        <Image src={editImage} alt="Preview" width={96} height={96}
className="w-full h-full object-cover" />
                    ) : (
                        <span className="material-symbols-outlined text-4xl text-gray-
400">add_a_photo</span>
                    )}
                </div>
                <button onClick={() => profileImageInputRef.current?.click()}
className="text-primary text-sm font-bold">Change Photo</button>
                <input ref={profileImageInputRef} type="file" accept="image/*"
className="hidden" onChange={handleProfileImageUpload} />
            </div>
            <div>
                <label className="block text-sm font-medium text-text-secondary mb-
1">Full Name</label>
                <input value={editName} onChange={e => setEditName(e.target.value)}
                    className="w-full h-10 rounded-lg border border-gray-300 px-3 text-
sm focus:border-primary focus:ring-1 focus:ring-primary outline-none" />
            </div>
            <div className="flex justify-end gap-3 mt-6">
                <button onClick={() => setShowEditProfile(false)} className="px-4 py-2
rounded-lg text-text-secondary font-semibold hover:bg-gray-100">Cancel</button>
                <button onClick={handleSaveProfile} className="px-4 py-2 rounded-lg
bg-primary text-white font-bold shadow-sm hover:bg-primary-dark">Save
Changes</button>
            </div>
        </div>
    </div>

```



```

    </Modal>

    <p className="text-center text-text-secondary text-xs pb-4">
      CablePro v1.0.5 | © 2026 CablePro Systems Ltd.
    </p>
  </div>
);
}

```

7.2 Application Layout (src/components/AppShell.tsx)

The AppShell component wraps all authenticated pages and provides the primary navigation structure. It renders a responsive sidebar for desktop devices and a fixed bottom navigation bar for mobile devices. The desktop sidebar contains the CablePro logo, navigation links for Dashboard, Customers, Payments, Reports, and Settings, and a user profile section at the bottom with logout option. The mobile bottom navigation uses a center FAB (+) button for the primary 'New Payment' action, flanked by Home, Customers, Reports, and Settings icons. The top header bar shows the current page title, a dark mode toggle, and the user's profile picture/initials.

```

import Link from "next/link";
import Image from "next/image";
import { usePathname, useRouter } from "next/navigation";
import { useState } from "react";
import { useStore } from "@lib/store";
import Modal from "@components/Modal";

const navItems = [
  { href: "/dashboard", label: "Dashboard", icon: "grid_view" },
  { href: "/customers", label: "Customers", icon: "group" },
  { href: "/payments", label: "Payments", icon: "account_balance_wallet" },
  { href: "/reports", label: "Reports", icon: "analytics" },
  { href: "/settings", label: "Settings", icon: "settings" },
];

export default function AppShell({ children }: { children: React.ReactNode }) {

```

```

const pathname = usePathname();
const router = useRouter();
const { currentUser, logout, theme, toggleTheme } = useStore();
const [sidebarOpen, setSidebarOpen] = useState(false);
const [showLogoutConfirm, setShowLogoutConfirm] = useState(false);

const handleLogout = () => {
  logout();
  router.push("/");
};

const confirmLogout = () => {
  setShowLogoutConfirm(true);
};

const initials = currentUser?.fullName
  ? currentUser.fullName.split(" ").map(w => w[0]).join("").toUpperCase().slice(0, 2)
  : "OP";

return (
  <div className="flex min-h-screen bg-bg-light dark:bg-bg-dark transition-colors
duration-200">
    <div /* Mobile Overlay */
    {sidebarOpen && (
      <div
        className="fixed inset-0 z-40 bg-black/40 backdrop-blur-sm lg:hidden"
        onClick={() => setSidebarOpen(false)}
      />
    )}

    <div /* Sidebar */
    <aside
      className={`fixed top-0 left-0 z-50 h-full w-64 bg-white dark:bg-bg-card-dark
border-r border-border dark:border-border-dark
flex flex-col transform transition-transform duration-250 ease-in-out
lg:translate-x-0 lg:sticky lg:top-0 lg:h-screen lg:z-auto
${sidebarOpen ? "translate-x-0" : "-translate-x-full"}`}
    >
      <div /* Logo */
      <div className="flex items-center gap-3 px-6 py-5 border-b border-border
shrink-0">
        <Image src="/logo.png" alt="CablePro" width={44} height={44}
className="rounded-lg shadow-sm" />
        <div>
          <h1 className="text-lg font-bold text-text-primary dark:text-white
tracking-tight">CablePro</h1>
          <p className="text-[11px] text-text-secondary dark:text-gray-400 font-
medium">Management System</p>

```

```

    </div>
  </div>

  {/ * Nav Items */}
  <nav className="flex flex-col gap-1 p-3 mt-2 flex-1">
    {navItems.map((item) => {
      const isActive = pathname.startsWith(item.href);
      return (
        <Link
          key={item.href}
          href={item.href}
          onClick={() => setSidebarOpen(false)}
          className={`flex items-center gap-3 px-4 py-3 rounded-xl text-sm
font-semibold transition-all duration-150
${isActive
? "bg-primary/10 text-primary dark:bg-primary/20 dark:text-
blue-300"
: "text-text-secondary hover:bg-bg-light hover:text-text-primary
dark:text-gray-400 dark:hover:bg-gray-800 dark:hover:text-white"}
        >
          <span className="material-symbols-outlined text-
xl">{item.icon}</span>
          {item.label}
        </Link>
      );
    })}
  </nav>

  {/ * Bottom section */}
  <div className="shrink-0 p-4 pb-24 lg:pb-4 border-t border-border
dark:border-border-dark bg-white dark:bg-bg-card-dark">
    <div className="flex items-center gap-3 px-2">
      {currentUser?.profileImage ? (
        <Image src={currentUser.profileImage} alt="Profile" width={36}
height={36} className="w-9 h-9 rounded-full object-cover" />
      ) : (
        <div className="w-9 h-9 rounded-full bg-primary/10 text-primary flex
items-center justify-center text-sm font-bold">
          {initials}
        </div>
      )}
      <div className="flex-1 min-w-0">
        <p className="text-sm font-semibold text-text-primary dark:text-
white truncate">{currentUser?.fullName || "User"}</p>
        <p className="text-xs text-text-secondary dark:text-gray-400
truncate">{currentUser?.email || ""}</p>
      </div>
    </div>
  </div>

```

```

        <button onClick={confirmLogout} title="Logout">
          <span className="material-symbols-outlined text-text-secondary
text-lg cursor-pointer hover:text-danger transition-colors">
            logout
          </span>
        </button>
      </div>
    </div>
  </aside>

  {/ * Main Content */}
  <div className="flex-1 flex flex-col min-w-0">
    {/ * Top Header */}
    <header className="sticky top-0 z-30 flex items-center justify-between bg-
white/80 dark:bg-bg-card-dark/80 backdrop-blur-md border-b border-border
dark:border-border-dark px-4 py-3 lg:px-6">
      <div className="flex items-center gap-3">
        <button
          className="lg:hidden flex items-center justify-center w-10 h-10
rounded-xl hover:bg-bg-light dark:hover:bg-gray-800 transition"
          onClick={() => setSidebarOpen(true)}
        >
          <span className="material-symbols-outlined text-text-primary
dark:text-white">menu</span>
        </button>
        <h2 className="text-lg font-bold text-text-primary dark:text-white
tracking-tight capitalize">
          {pathname.split("/").pop() || "Dashboard"}
        </h2>
      </div>
      <div className="flex items-center gap-2">
        <button
          onClick={(e) => {
            e.stopPropagation();
            toggleTheme();
          }}
          className="flex items-center justify-center w-10 h-10 rounded-xl
hover:bg-gray-100 dark:hover:bg-gray-800 transition-colors cursor-pointer z-50
relative"
          title={theme === "light" ? "Switch to Dark Mode" : "Switch to Light
Mode"}
          aria-label="Toggle Theme"
        >
          <span className="material-symbols-outlined text-text-secondary
dark:text-gray-400 text-2xl">
            {theme === "light" ? "dark_mode" : "light_mode"}
          </span>
        </button>
      </div>
    </div>
  </div>

```

```

        <Link href="/settings" className="flex items-center gap-2 ml-1">
            {currentUser?.profileImage ? (
                <Image src={currentUser.profileImage} alt="Profile" width={36}
height={36} className="w-9 h-9 rounded-full object-cover border border-gray-200
dark:border-gray-700" />
            ) : (
                <div className="w-9 h-9 rounded-full bg-primary/10 text-primary
flex items-center justify-center text-sm font-bold border border-primary/20">
                    {initials}
                </div>
            )}
        </Link>
    </div>
</header>

    /* Page Content — extra bottom padding on mobile for bottom nav */
    <main className="flex-1 p-4 lg:p-6 pb-24 lg:pb-6">
        {children}
    </main>
</div>

    /* — Mobile Bottom Navigation Bar (matches reference) — */
    <nav className="fixed bottom-0 left-0 right-0 bg-white dark:bg-bg-card-dark
border-t border-gray-200 dark:border-border-dark px-6 py-3 flex justify-between items-
center lg:hidden"
        style={{ zIndex: 9999, paddingBottom: "max(env(safe-area-inset-bottom,
0px), 12px)" }}>
        <Link href="/dashboard"
            className={`flex flex-col items-center gap-1
${pathname.startsWith("/dashboard")} ? "text-primary" : "text-gray-400"}`
            style={{ touchAction: "manipulation", WebkitTapHighlightColor:
"transparent" }}>
            <span className="material-symbols-outlined">grid_view</span>
            <span className={text-[10px] ${pathname.startsWith("/dashboard")} ?
"font-bold" : "font-medium"}}>Home</span>
        </Link>

        <Link href="/customers"
            className={`flex flex-col items-center gap-1
${pathname.startsWith("/customers")} ? "text-primary" : "text-gray-400"}`
            style={{ touchAction: "manipulation", WebkitTapHighlightColor:
"transparent" }}>
            <span className="material-symbols-outlined">group</span>
            <span className={text-[10px] ${pathname.startsWith("/customers")} ?
"font-bold" : "font-medium"}}>Customers</span>
        </Link>

        <div className="-mt-12">

```

```

    <Link href="/payments/new"
      className="flex w-14 h-14 items-center justify-center rounded-full bg-
primary text-white shadow-lg border-4 border-bg-light dark:border-bg-dark"
      style={{ touchAction: "manipulation", WebkitTapHighlightColor:
"transparent" }}>
      <span className="material-symbols-outlined text-3xl">add</span>
    </Link>
  </div>

  <Link href="/reports"
    className={`flex flex-col items-center gap-1
${pathname.startsWith("/reports")} ? "text-primary" : "text-gray-400"}`
    style={{ touchAction: "manipulation", WebkitTapHighlightColor:
"transparent" }}>
    <span className="material-symbols-outlined">description</span>
    <span className={`text-[10px] ${pathname.startsWith("/reports")} ? "font-
bold" : "font-medium"}`>Reports</span>
  </Link>

  <Link href="/settings"
    className={`flex flex-col items-center gap-1
${pathname.startsWith("/settings")} ? "text-primary" : "text-gray-400"}`
    style={{ touchAction: "manipulation", WebkitTapHighlightColor:
"transparent" }}>
    <span className="material-symbols-outlined">settings</span>
    <span className={`text-[10px] ${pathname.startsWith("/settings")} ? "font-
bold" : "font-medium"}`>Settings</span>
  </Link>
</nav>

<Modal isOpen={showLogoutConfirm} onClose={() =>
setShowLogoutConfirm(false)} title="Confirm Logout">
  <p className="text-text-secondary mb-6">Are you sure you want to log
out?</p>
  <div className="flex justify-end gap-3">
    <button onClick={() => setShowLogoutConfirm(false)} className="px-4
py-2 rounded-lg text-text-secondary font-semibold hover:bg-gray-
100">Cancel</button>
    <button onClick={handleLogout} className="px-4 py-2 rounded-lg bg-
primary text-white font-bold shadow-sm hover:bg-primary-dark">Log Out</button>
  </div>
</Modal>
</div>
}

```

```

import { useEffect } from "react";
import { useRouter, usePathname } from "next/navigation";

```

```

import AppShell from "@components/AppShell";
import { useStore } from "@lib/store";

export default function AppLayout({
  children,
}: {
  children: React.ReactNode;
}) {
  const { currentUser, hydrated } = useStore();
  const router = useRouter();
  const pathname = usePathname();

  useEffect(() => {
    if (hydrated && !currentUser) {
      router.replace("/");
    }
  }, [currentUser, router, pathname, hydrated]);

  if (!currentUser) {
    return (
      <div className="min-h-screen flex items-center justify-center bg-bg-light">
        <div className="animate-pulse text-primary font-bold text-
lg">Redirecting...</div>
      </div>
    );
  }

  return <AppShell>{children}</AppShell>;
}

```

SCREENSHOTS

8.SCREENSHOTS

8.1 Login Page

The login page features a gradient background transitioning from indigo to white, with a centered form containing the CablePro branding, email/username field, password field with visibility toggle, 'Remember me' checkbox, 'Forgot Password' link, and Google Sign-In button.

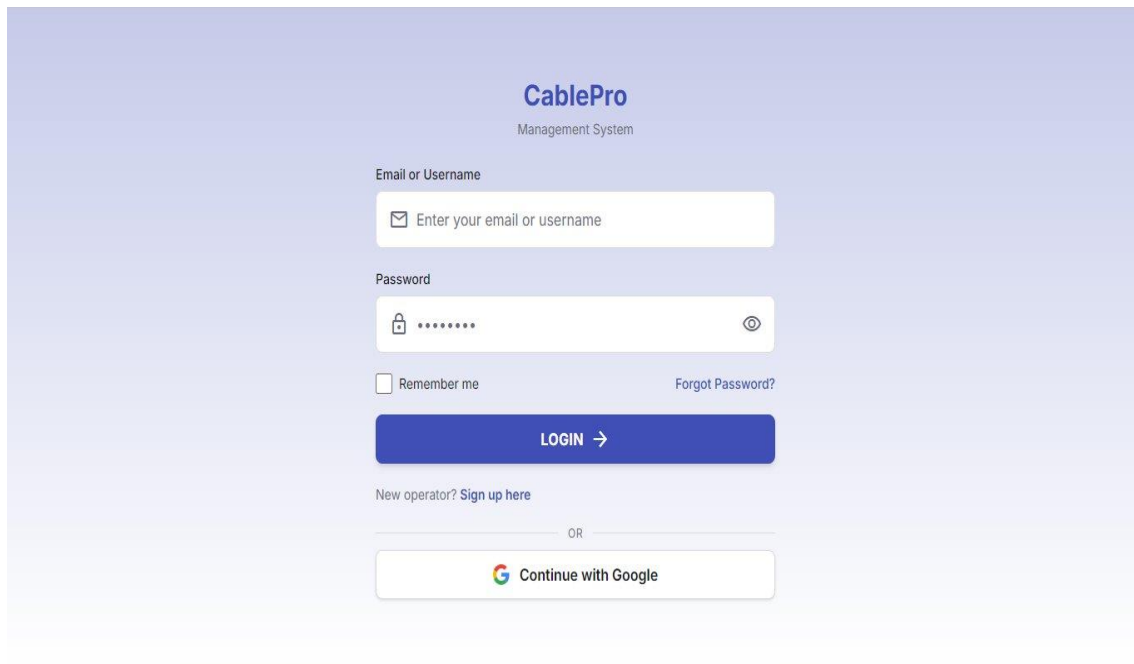


Figure 9.1: CablePro Login Page

8.2 Dashboard

The dashboard provides a complete business overview with greeting, date display, four KPI cards (Total Customers: 8, Active: 7, Outstanding: ₹300, Total Paid: 6), quick action buttons, a weekly collection bar chart showing ₹2,650 total, and a payment methods donut chart showing UPI (₹2,200) and Cash (₹1,200) distribution.

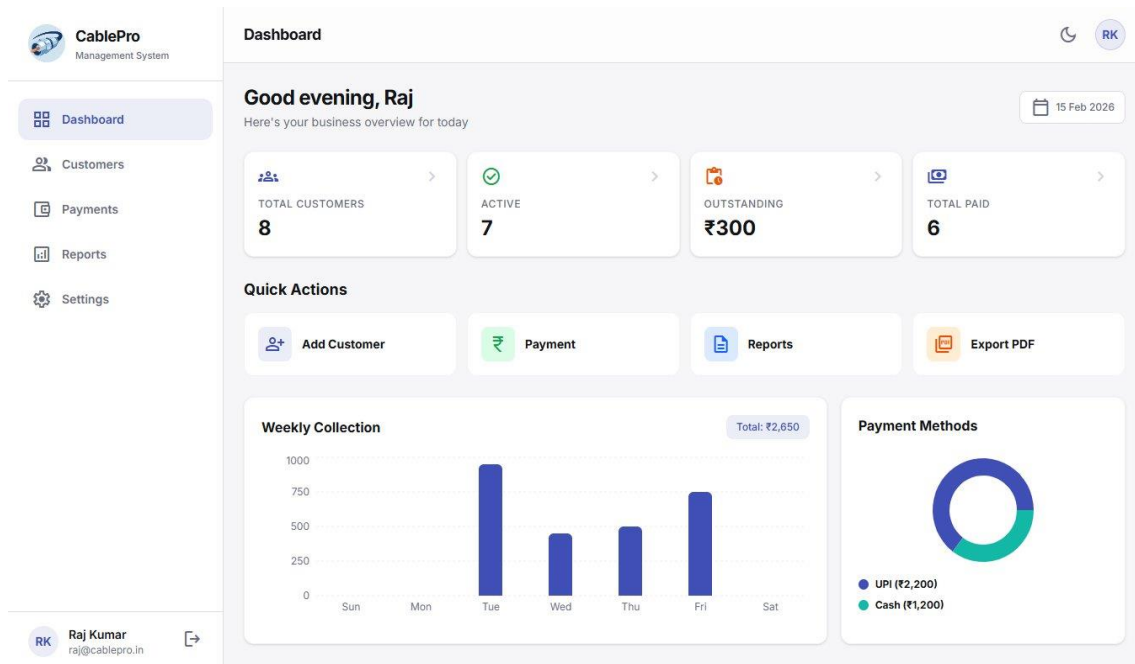


Figure 9.2: Dashboard with Analytics and Quick Actions

8.3 Customers Page

The customers page shows the subscriber list with search functionality, status filter tabs (All, Active, Deactive, Overdue, Paid), and customer count display. The 'No customers found' empty state with icon is shown when no records match the filter. The floating '+' button (teal) in the bottom right opens the Add Customer modal.

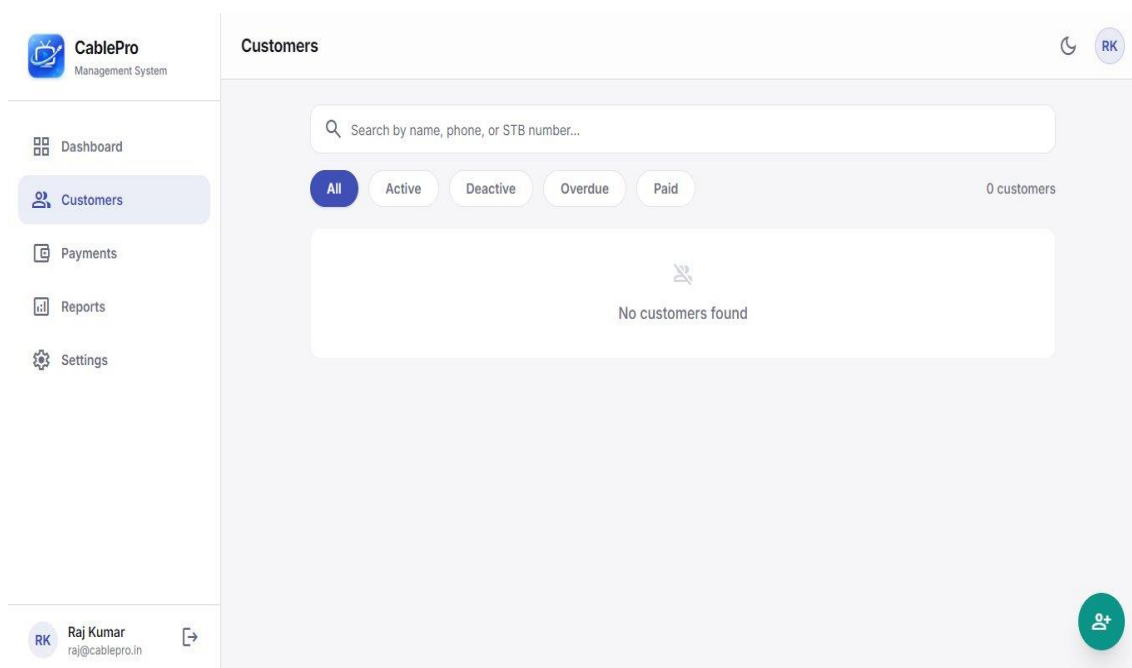


Figure 9.3: Customers Management Page

8.4 Payments Page

The payments page displays collection summary cards for Today (₹0), This Week (₹0), and This Month (₹0) at the top. The payment list shows all transactions with customer name, date, method, amount, status badge, and action buttons. The blue 'New Payment' FAB is fixed in the bottom right corner.

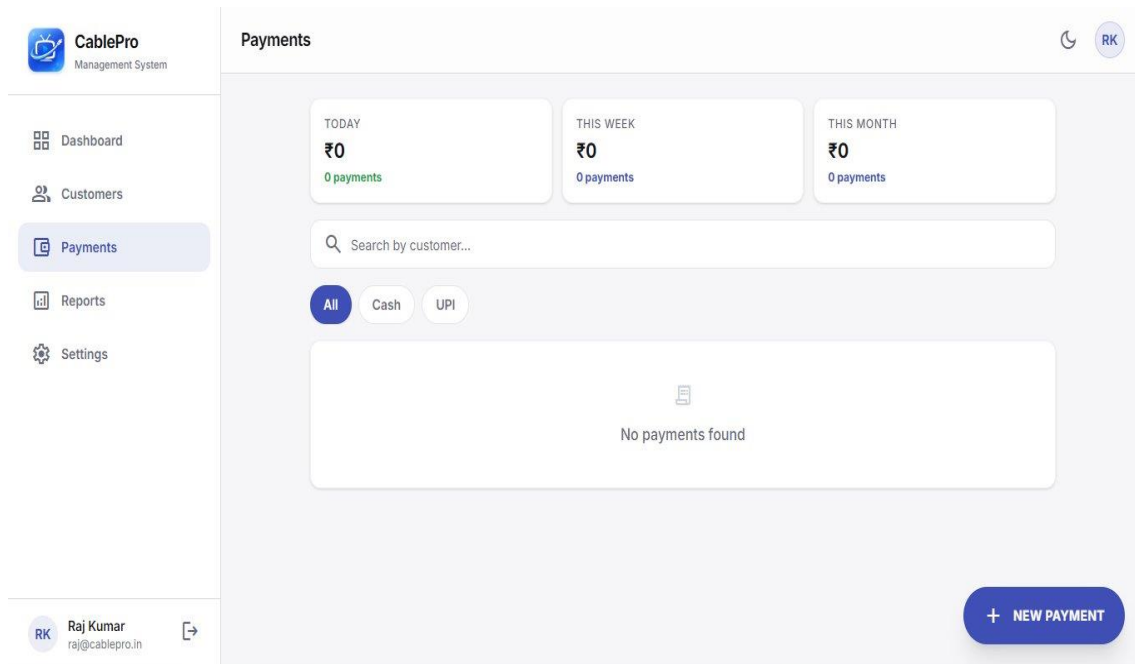


Figure 9.4: Payments Management Page

8.5 Reports Page

The reports page provides financial analytics with month/custom date range filtering, KPI cards (Total Collection: ₹0, Transactions: 0, Avg. Transaction: ₹0), Collection Trend area chart, Payment Methods breakdown, and Daily Breakdown table. Export CSV and Export PDF buttons are in the top right corner.

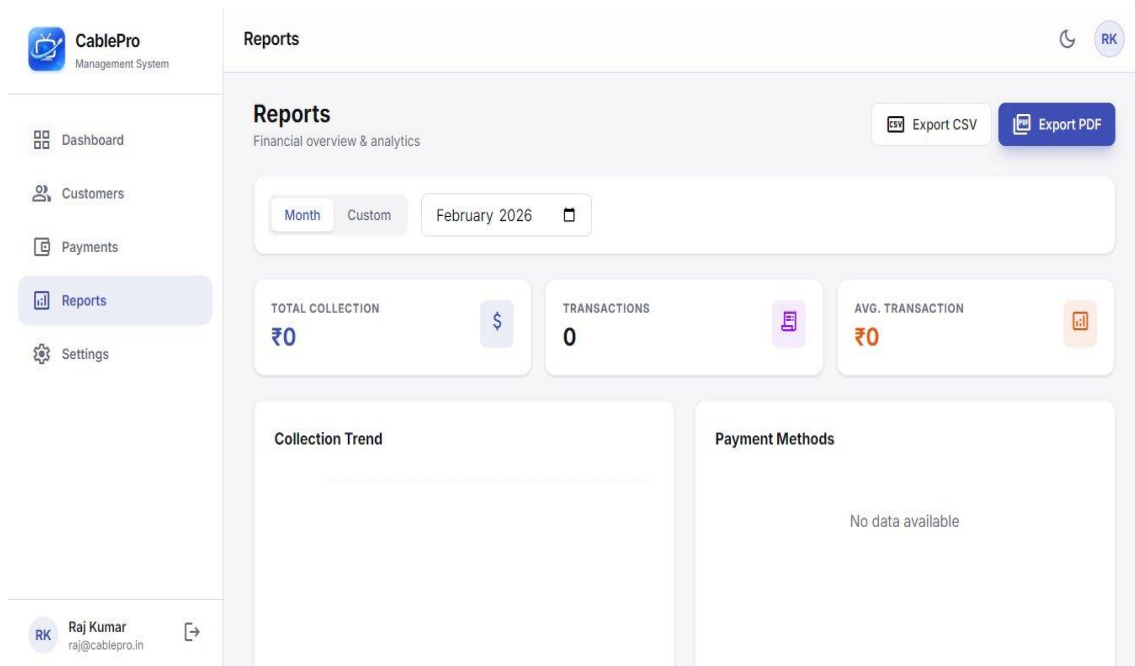


Figure 9.5: Reports and Analytics Page

8.6 Settings Page

The settings page shows the operator's profile section with name (Raj Kumar), email (raj@cablepro.in), and role (Operator). The Data Summary section displays Total Customers (0), Total Payments (0), and Data Storage (Local Browser). Below are the Import/Export and Cloud Backup sections.

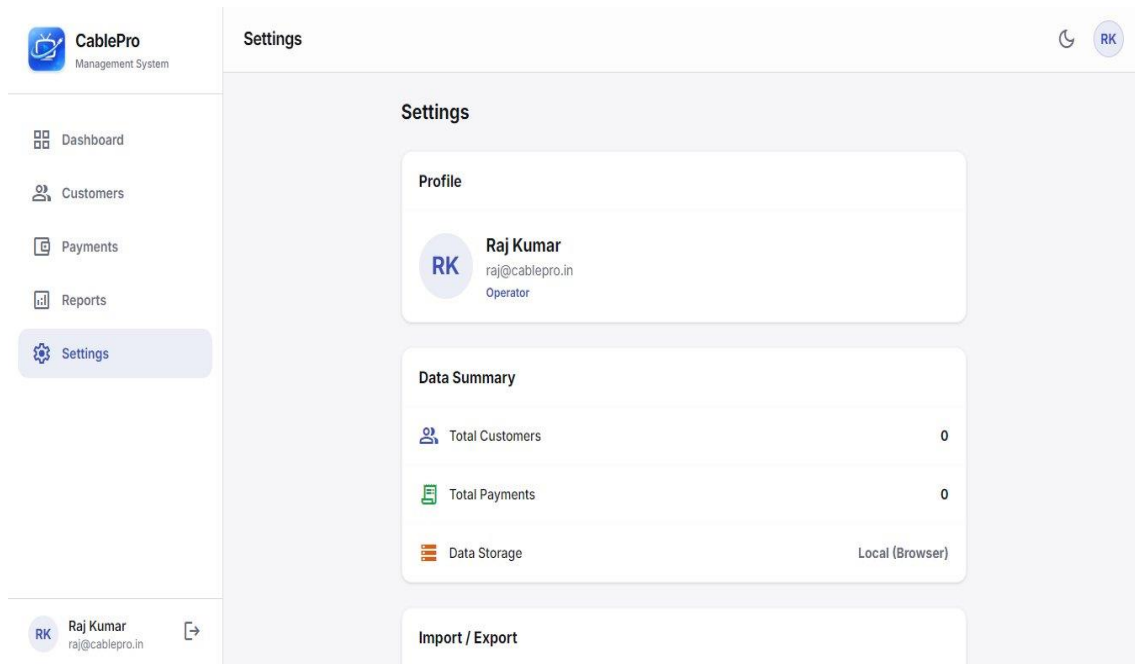


Figure 9.6: Settings Page

TESTING

9.TESTING

Testing is a critical phase in the software development lifecycle that ensures the application functions correctly, meets requirements, and provides a quality user experience. CablePro underwent comprehensive testing covering unit, integration, system, and user acceptance testing.

9.1 Unit Testing

Unit testing focuses on testing individual functions and components in isolation. The following business logic functions were unit tested:

9.1.1 Outstanding Calculation Tests

Test Case	Input	Expected Output
Deactive customer	status: 'Deactive'	Outstanding: ₹0
No payment history	0 confirmed payments	Outstanding: monthlyRate
Payment within 1-month plan	Last pay: 2026-02-01, today: 2026-02-15	Outstanding: ₹0
Payment expired 1-month plan	Last pay: 2026-01-01, today: 2026-02-15	Outstanding: monthlyRate
Payment within 3-month plan	Last pay: 2026-01-01, today: 2026-03-15	Outstanding: ₹0
Payment expired 3-month plan	Last pay: 2025-10-01, today: 2026-02-15	Outstanding: monthlyRate
Cancelled payment (not counted)	Last pay cancelled	Outstanding: monthlyRate

9.1.2 Customer Validation Tests

Test Case	Input	Expected Result
Valid customer	All fields valid, unique	Customer added successfully
Missing name	name: "	Error: Customer Name is required.

Invalid phone (9 digits)	phone: '987654321'	Error: Mobile Number must be exactly 10 digits.
Duplicate phone	phone already exists	Error: Mobile Number already exists.
Non-numeric STB	stbNumber: 'ABC123'	Error: STB Number must be numeric only.
Duplicate STB	STB already exists	Error: STB Number already exists.

9.2 Integration Testing

Integration testing verifies that different modules work together correctly. Key integration test scenarios:

9.2.1 Authentication + Data Loading Integration

Test Scenario	Steps	Expected Result
Login loads correct data	Login as operator A, check customers	Only operator A's customers shown
Logout clears data	Login, verify data, logout	Data cleared from state
Switch users	Login A, logout, login B	B's data loaded (not A's)
New user gets empty state	Login new Firebase user	Empty customers/payments arrays
Seed user gets demo data	Login as 'operator'	8 customers, 9 payments loaded

9.2.2 Payment + Dashboard Integration

Test Scenario	Expected Behavior
Record payment → Dashboard updates	Today's collection, weekly chart, and KPIs refresh
Cancel payment → Outstanding recalculates	Customer card shows outstanding again
Add customer → Total Customers KPI updates	Dashboard KPI shows incremented count

Deactivate customer → Active count updates	Active KPI decremented
--	------------------------

9.3 System Testing

System testing validates the complete application against functional and non-functional requirements:

9.3.1 Functional Testing

Feature	Test Result	Status
Email/Password Login	Successfully authenticates valid users	PASS
Google OAuth Login	OAuth popup, token capture, redirect works	PASS
Add Customer with validation	All validation rules enforced correctly	PASS
Edit Customer	Existing data pre-populated, changes saved	PASS
Soft Delete Customer	Customer hidden but data retained	PASS
Record Payment (Cash/UPI)	Payment saved, dashboard updates	PASS
Duplicate payment prevention	Blocked with clear error message	PASS
Cancel payment	Payment status updated, outstanding recalculated	PASS
Search customers	Real-time filter by name/phone/STB	PASS
Status filter tabs	Correct customers shown for each filter	PASS
Report month filter	Only payments from selected month shown	PASS
Report custom date range	Payments filtered by date range	PASS
Export CSV (customers)	Valid CSV file downloaded	PASS
Export PDF (reports)	Print dialog opens with formatted report	PASS

Google Drive backup	JSON file created/updated in Drive	PASS
Google Drive restore	Data restored from Drive JSON	PASS
Import customers CSV	Customers parsed and added from CSV	PASS
Dark mode toggle	Interface switches dark/light correctly	PASS
Profile image upload	Image converted to Base64, displayed	PASS

9.3.2 Non-Functional Testing

Aspect	Test	Result
Performance	Page load time on 4G network	Under 3 seconds initial load
Mobile Responsiveness	Tested on 360px, 768px, 1280px widths	Fully responsive at all widths
Offline Capability	Disabled network, navigated pages	All data accessible offline
Data Persistence	Closed browser, reopened	All data retained in localStorage
Cross-browser	Tested Chrome, Firefox, Safari, Edge	Consistent behavior all browsers
PWA installability	Add to Home Screen tested	Works as PWA on Android/iOS
Security	Attempted direct URL access without auth	Redirected to login page
Input Sanitization	Special chars in customer name	Safely stored and displayed

9.4 User Acceptance Testing

User Acceptance Testing (UAT) was conducted with simulated cable TV operators to verify that CablePro meets real-world usability requirements. The test scenarios and feedback are summarized below:

UAT Scenario	User Feedback	Resolution
Add 5 customers	Easy to use, form is intuitive	No changes needed

Record payments for all	Fast, auto-fills amount from plan	Feature confirmed useful
Find overdue customers	Filter tab works great	No changes needed
Export monthly report	PDF export via print is clear	Added export instructions
Use on mobile phone	Very smooth, bottom nav works well	No changes needed
Dark mode in low light	Much easier on eyes at night	Feature well-received
Backup to Google Drive	Two-tap backup is convenient	No changes needed

The UAT sessions confirmed that CablePro is suitable for real-world deployment, with positive feedback on mobile usability, payment recording speed, and the outstanding balance visibility on customer cards.

CONCLUSION

10.CONCLUSION

CablePro is a comprehensive, modern, and user-friendly Cable TV Management System that successfully addresses the real operational challenges faced by cable TV operators in India. The system was developed using cutting-edge web technologies — Next.js 14, React 18, TypeScript, Tailwind CSS v4, and Firebase — resulting in a production-ready application that demonstrates the practical application of modern full-stack web development concepts.

The project achieves all of its stated objectives. The subscriber management system effectively handles complete customer lifecycle management with STB number tracking, plan duration management (1, 3, 6, and 12-month plans), and automatic outstanding balance calculation. The payment management module provides efficient transaction recording with duplicate prevention logic that respects multi-month billing cycles. The financial reporting module generates insightful analytics with export capabilities that meet real accounting needs.

The mobile-first Progressive Web App design ensures that cable TV operators can use CablePro effectively from their smartphones while visiting customers in the field — a critical requirement for the target users. The PWA capability combined with localStorage persistence enables the application to work even without internet connectivity, a significant advantage for operators in areas with unreliable network coverage.

The Firebase Authentication integration with Google OAuth and the Google Drive backup functionality demonstrate sophisticated integration of third-party APIs. The secure multi-method authentication ensures that business data remains protected while providing convenient access options. The cloud backup feature protects against data loss and enables future multi-device access.

The dark mode support, responsive design, intuitive navigation, and polished UI demonstrate attention to user experience that goes beyond functional requirements. These features reflect the real-world standards expected of professional software applications.

CablePro successfully bridges the gap between the basic tools (paper registers, spreadsheets) currently used by small cable TV operators and the expensive proprietary software available to larger operators. By providing a free, web-based, mobile-friendly alternative with professional-grade features, CablePro democratizes access to modern business management technology for the cable TV industry.

Limitations of the current system include: dependence on browser localStorage limits the total data storage (typically 5-10 MB per origin), meaning operators with very large subscriber bases (1000+) may need to migrate to Firestore; the application requires internet access for authentication, backup, and restore operations; and the Google Drive backup requires a Google account, which not all operators may have.

Despite these limitations, CablePro represents a significant achievement in terms of technical complexity, user experience design, and real-world applicability. The system is ready for deployment and use by cable TV operators of small to medium scale.

FUTURE ENHANCEMENTS

11.FUTURE ENHANCEMENTS

CablePro's modular architecture provides a strong foundation for future enhancements. The following improvements are planned for subsequent versions:

11.1 Short-Term Enhancements (Version 2.0)

11.1.1 Firebase Firestore Integration

Migrating the primary data store from browser localStorage to Firebase Firestore will enable real-time multi-device synchronization, eliminating the need for manual cloud backups. Operators will be able to access and update their subscriber data simultaneously from multiple devices (e.g., a desktop at the office and a smartphone in the field), with changes reflecting in real-time across all devices.

11.1.2 WhatsApp Payment Reminders

Integration with the WhatsApp Business API or WhatsApp direct URL scheme will enable operators to send automated payment reminder messages to subscribers who are overdue. The customer detail page already has a WhatsApp button — this enhancement will generate a pre-filled reminder message template. Bulk reminder sending for all overdue customers will also be implemented.

11.1.3 Digital Payment Receipt

Generating professional digital receipts in PDF format for each payment transaction. Receipts will include the operator's business name, customer name, STB number, payment date, amount, payment method, billing period, and a unique receipt number. Receipts can be shared via WhatsApp or downloaded directly.

11.1.4 SMS Notifications

Integration with SMS gateway APIs (such as Twilio or MSG91) to send automated SMS notifications for payment confirmations, due date reminders, and service activation/deactivation alerts. Bulk SMS capability for monthly billing reminders will help operators improve payment collection rates.

12.2 Medium-Term Enhancements (Version 3.0)

11.2.1 Multi-Operator/Staff Support

Implementing role-based access control (RBAC) to allow cable TV businesses with multiple staff members to use CablePro. The admin role will have full access to all features and analytics, while a collector role will be limited to recording payments and viewing assigned customers. Data isolation between staff members will be maintained while providing management oversight.

11.2.2 Automated Monthly Billing

Implementing an automated billing system that generates monthly bills at the start of each billing cycle, automatically marks customers as overdue when their plan expires, sends automated reminders at configurable intervals before and after the due date, and tracks partial payments with running balance.

11.2.3 UPI Payment Integration

Integration with UPI payment gateways (Razorpay, PhonePe Business) to allow operators to generate UPI payment links that customers can use to pay directly from their phones. Payment confirmation will automatically update the system, eliminating manual payment entry.

11.2.4 Customer Mobile App

A customer-facing mobile application that allows cable TV subscribers to view their account status, outstanding dues, payment history, and plan details. Customers can also make UPI payments directly through the app, reducing the operator's manual collection burden.

11.3 Long-Term Enhancements (Version 4.0)

11.3.1 AI-Powered Analytics

Implementing machine learning models to predict subscriber churn (likelihood of disconnection), identify payment default patterns, forecast monthly collection amounts, and recommend optimal pricing strategies for different areas. AI-driven insights will help operators make proactive business decisions.

11.3.2 Multi-Language Support

Adding support for regional Indian languages including Tamil, Telugu, Hindi, Malayalam, and Kannada. The interface will be fully translatable, allowing operators across different states to use CablePro in their preferred language.

11.3.3 TRAI Compliance Reporting

Generating regulatory compliance reports as required by the Telecom Regulatory Authority of India (TRAI) for digital cable TV operations. This includes subscriber count reports, STB inventory reports, and channel pack distribution reports in the required formats.

11.3.4 Technician Field App

A separate application for field technicians (cable installation and repair staff) that integrates with the operator's CablePro account. Technicians can view service requests, update STB installation status, capture installation photos, and collect payments — all linked to the operator's subscriber database.

REFERENCES

12.REFERENCES

12.1 Online Documentation

1. Next.js Documentation — Official Next.js App Router documentation, Vercel Inc.
<https://nextjs.org/docs>
2. React Documentation — Official React.js documentation and API reference.
<https://react.dev>
3. TypeScript Documentation — Official TypeScript language reference.
<https://www.typescriptlang.org/docs>
4. Tailwind CSS Documentation — Official Tailwind CSS utility classes reference.
<https://tailwindcss.com/docs>
5. Firebase Authentication Documentation — Google Firebase Auth reference.
<https://firebase.google.com/docs/auth>
6. Google Drive API v3 Documentation — Google Developers Drive API reference.
<https://developers.google.com/drive/api/v3/reference>
7. Recharts Documentation — React charting library documentation.
<https://recharts.org/en-US/api>
8. MDN Web Docs — Web technology reference (HTML, CSS, JS, Web APIs).
<https://developer.mozilla.org>
9. Google Material Symbols — Icon library documentation.
<https://fonts.google.com/icons>
10. Vercel Documentation — Deployment platform documentation.
<https://vercel.com/docs>

12.2 Academic References

11. Flanagan, D. (2020). JavaScript: The Definitive Guide, 7th Edition. O'Reilly Media.
12. Freeman, A. (2022). Pro React 16. Apress.
13. Wieruch, R. (2022). The Road to React. Self-published.

14. Norman, D. (2013). The Design of Everyday Things: Revised and Expanded Edition. Basic Books.
15. Nielsen, J. (1994). Usability Engineering. Morgan Kaufmann.

12.3 Technical Articles

16. "Building Progressive Web Apps" — Google Developers Web Fundamentals.
<https://web.dev/progressive-web-apps>
17. "localStorage vs Firestore: When to Use Each" — Firebase Blog.
<https://firebase.google.com/support/faq>
18. "React Context vs Redux for State Management" — Kent C. Dodds Blog.
<https://kentcdodds.com/blog>
19. "Web Accessibility Guidelines (WCAG 2.1)" — W3C.
<https://www.w3.org/WAI/WCAG21/quickref>
20. "India Cable TV Digitization — TRAI Report 2023" — Telecom Regulatory Authority of India. <https://www.trai.gov.in>

12.4 Development Community Resources

21. Stack Overflow — Developer Q&A community for troubleshooting.
<https://stackoverflow.com>
22. GitHub — Version control and code repository. <https://github.com>
23. CSS Tricks — CSS and web design techniques. <https://css-tricks.com>
24. GeeksforGeeks — Algorithms, data structures, and web development tutorials.
<https://www.geeksforgeeks.org>
25. W3Schools — Web development learning resource. <https://www.w3schools.com>

BIBLIOGRAPHY

13.BIBLIOGRAPHY

The following sources were consulted and referenced during the design, development, and documentation of the CablePro Cable TV Management System:

13.1 Textbooks and Printed References

26. Crockford, D. (2008). JavaScript: The Good Parts. O'Reilly Media, Sebastopol, CA. ISBN: 978-0-596-51774-8.
27. Haverbeke, M. (2018). Eloquent JavaScript: A Modern Introduction to Programming, 3rd Edition. No Starch Press.
28. Matthes, E. (2019). Python Crash Course, 2nd Edition. No Starch Press. (Referenced for backend scripting concepts)
29. Hunt, A. and Thomas, D. (1999). The Pragmatic Programmer. Addison-Wesley. (Referenced for software engineering principles)
30. Gamma, E., Helm, R., Johnson, R., and Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.

13.2 Online APIs and Services Used

31. Firebase Authentication (Google LLC) — User authentication service. Used for email/password and OAuth authentication. <https://firebase.google.com>
32. Google Drive API v3 (Google LLC) — Cloud file storage API. Used for backup and restore functionality. <https://developers.google.com/drive>
33. Google Fonts API — Web font and icon service. Used for Inter font and Material Symbols icons. <https://fonts.google.com>
34. Vercel Platform — Serverless hosting and deployment. Used for production deployment. <https://vercel.com>
35. GitHub — Source code hosting and version control. <https://github.com>

13.3 Open Source Libraries Used

Library	Version	License
Next.js	14.x	MIT License
React	18.x	MIT License
TypeScript	5.x	Apache 2.0
Tailwind CSS	4.x	MIT License
Firebase SDK	10.x	Apache 2.0
Recharts	Latest	MIT License
docx (document gen.)	Latest	MIT License

13.4 Development Tools

36. Visual Studio Code (Microsoft) — Primary IDE for development.

<https://code.visualstudio.com>

37. Git (Linus Torvalds) — Version control system. <https://git-scm.com>

38. Node.js (OpenJS Foundation) — JavaScript runtime environment. <https://nodejs.org>

39. Figma (Figma Inc.) — UI/UX design and prototyping tool. <https://figma.com>

40. Google Chrome DevTools — Browser developer tools for debugging and testing.

<https://developer.chrome.com/docs/devtools>

41. Postman — API testing and documentation tool. <https://www.postman.com>